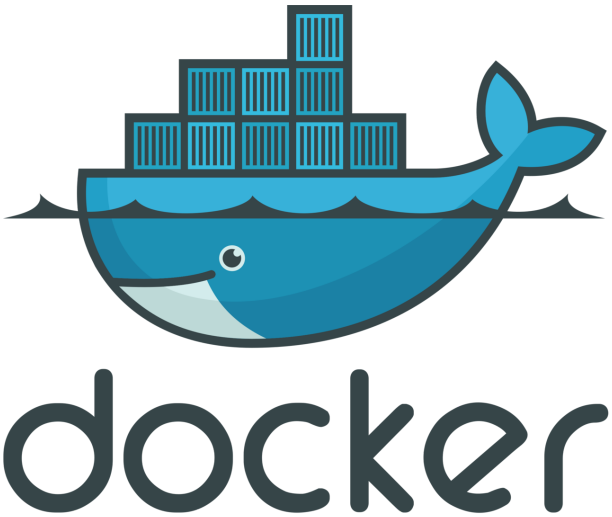

Manual de Docker para Principiantes

En Windows 11: De cero a despliegue

Guía de supervivencia para entornos cliente/servidor

© Autor: **Agustín Crespo Valencia**

Índice

1. Introducción: ¿Qué es esto y para qué sirve? 1
1.1. De los Barcos a los Food Trucks: La Analogía . . . 1
1.2. Máquinas Virtuales vs. Docker: ¿Por qué Docker es más ligero? . . . 1
1.3. ¿Qué problema resuelve? (El famoso *"En mi máquina funcionaba"*) . . . 1

2. Conceptos Básicos (explicados "para torpes") 2
2.1. Imagen (Image): El plano o la receta de cocina . . . 2
2.2. Contenedor (Container): El pastel cocinado . . . 2
2.3. Docker Hub: El supermercado de ingredientes . . . 2
2.4. Cliente vs. Servidor: Quién manda y quién obedece . . . 2

3. Preparando el terreno: Instalación en Windows 11 3
3.1. ¿Qué es WSL2 y por qué Windows necesita un "corazón de Linux"? . . . 3
3.2. Instalación paso a paso de Docker Desktop . . . 3
3.3. Tour por la interfaz gráfica de Docker Desktop . . . 5
3.4. Configuración básica de recursos (RAM y CPU) . . . 11

4. Primeros pasos: Hola Mundo 13
4.1. Tu primera orden en la terminal (PowerShell) . . . 13
4.2. Entendiendo qué ha pasado "por detrás" . . . 13
4.3. El ciclo de vida: Crear, Iniciar, Parar y Borrar . . . 14
4.3.1. Crear y dejar trabajando en segundo plano (`-d`) . . . 14
4.3.2. Ver los locales abiertos (`docker ps`) . . . 14
4.3.3. Cerrar el local por hoy (`docker stop`) . . . 15
4.3.4. Volver a abrir al día siguiente (`docker start`) . . . 15
4.3.5. Demoler el local (`docker rm`) . . . 16

5. Redes para quien no sabe de Redes 17
5.1. El concepto de aislamiento: Tu contenedor es una caja fuerte . . . 17
5.2. Conceptos previos: ¿Qué es una IP y qué es un Puerto? . . . 17
5.2.1. La Dirección IP (La calle y el número) . . . 17
5.2.2. El Puerto (La puerta o ventanilla específica) . . . 17
5.3. El problema de los choques en Windows . . . 18
5.4. El comando `-p` (Mapeo de puertos): Construyendo el puente . . . 18
5.5. ¿Qué es `localhost`? Accediendo a tu aplicación . . . 18

6. Almacenamiento: Cómo no perder tus datos 20
6.1. El problema: Los contenedores son efímeros . . . 20
6.2. Volúmenes: La caja fuerte gestionada por Docker . . . 20
6.3. Bind Mounts: La cinta transportadora hacia tu Escritorio . . . 20

7. Docker Compose: La Receta Maestra 22
7.1. ¿Por qué escribir comandos largos es mala idea? . . . 22
7.2. El archivo `docker-compose.yml`: Tu infraestructura como código . . . 22
7.3. Levantar y tirar todo el sistema con un solo comando . . . 23

8. Práctica Final: Montando un Servidor Web Real 24
8.1. Objetivo: Tu página web en el navegador . . . 24
8.2. Paso 1: Crear el archivo HTML en Windows (Tu receta) . . . 24
8.3. Paso 2: Configurar el Plano Maestro (docker-compose.yml) . . . 24
8.4. Paso 3: Conectar todo y ver el resultado . . . 25

9. Anexos y Chuletas **27**
9.1. Chuleta de comandos esenciales de supervivencia 27
9.2. Solución de problemas comunes en Windows 11 27
9.2.1. Error 1: “bind: address already in use” 27
9.2.2. Error 2: Docker Desktop se queda en “Starting...” y nunca arranca 28

1. Introducción: ¿Qué es esto y para qué sirve?

Si estás leyendo este manual, probablemente hayas escuchado que “Docker es el futuro” o que “todo el mundo usa contenedores hoy en día”.

Sin embargo, cuando intentas aprender, te encuentras con un muro de terminología compleja sobre virtualización, redes y núcleos de Linux.

Vamos a derribar ese muro. Olvídate de los conceptos informáticos por un momento.

1.1. De los Barcos a los Food Trucks: La Analogía

El nombre y el logo de Docker (una ballena llevando cajas) provienen de la **industria del transporte marítimo**. Antes, cargar un barco era un caos: sacos de café mezclados con cajas de herramientas y barriles de aceite. Hasta que se inventó el **contenedor estándar**. De repente, no importaba lo que hubiera dentro; el contenedor siempre tenía el mismo tamaño, encajaba en cualquier barco, tren o camión, y su interior estaba aislado.

Docker hace exactamente lo mismo, pero con aplicaciones informáticas (programas, bases de datos, páginas web, etcétera).

Para entender cómo funciona a lo largo de este manual, vamos a utilizar una analogía más cercana: **El modelo de Franquicias y los Food Trucks**.

Imagina que eres un emprendedor hostelero. Tienes una receta increíble para hacer hamburguesas y quieres abrir restaurantes por toda la ciudad (tu ordenador Windows 11).

1.2. Máquinas Virtuales vs. Docker: ¿Por qué Docker es más ligero?

Hasta hace poco, la forma tradicional de aislar aplicaciones era usar **Máquinas Virtuales** (como VirtualBox o VMware). Siguiendo nuestra analogía: Crear una Máquina Virtual es como **construir un restaurante entero de ladrillo desde cero**. Tienes que comprar el terreno, poner los cimientos, la instalación eléctrica, el agua y contratar a todo el personal de seguridad y mantenimiento. Es muy costoso, ocupa mucho espacio y tarda meses en abrir.

Usar Docker es como comprar un **Food Truck** totalmente equipado. El Food Truck utiliza las carreteras y la infraestructura de la ciudad que ya existe (el núcleo de tu propio sistema operativo). Solo lleva dentro exactamente lo que necesita para hacer hamburguesas: la plancha, los ingredientes y el cocinero. Por eso, mientras una máquina virtual puede pesar gigabytes y tardar minutos en encenderse, un contenedor de Docker pesa megabytes y arranca en milisegundos.

1.3. ¿Qué problema resuelve? (El famoso *“En mi máquina funcionaba”*)

Todo programador ha vivido esta pesadilla: Creas un programa en tu ordenador, funciona perfectamente, se lo pasas a un compañero (o lo subes al servidor de la empresa) y de repente falla todo. Las excusas habituales son: *“Es que tú tienes otra versión de Java”*, *“Es que en mi Windows funciona distinto que en tu Mac”*.

Docker elimina este problema de raíz. Cuando envías tu aplicación, no envías solo el código. Envías el Food Truck entero con la cocina montada, la temperatura ajustada y el cocinero dentro.

Si funciona en tu máquina, funcionará exactamente igual en cualquier otra parte del mundo que tenga Docker instalado.

2. Conceptos Básicos (explicados “para torpes”)

Antes de instalar nada, necesitamos dominar el vocabulario. Docker utiliza cuatro conceptos fundamentales que formarán tu día a día.

2.1. Imagen (Image): El plano o la receta de cocina

Una Imagen es un archivo de solo lectura. No hace nada por sí misma, no se ejecuta. Es, literalmente, el **Manual de la Franquicia**. Este manual contiene absolutamente todo: los planos del camión, las recetas exactas, qué temperatura debe tener la nevera y las instrucciones paso a paso para el cocinero. Tú no te puedes “comer” un manual, pero sin él, no puedes abrir el negocio.

2.2. Contenedor (Container): El pastel cocinado

Un Contenedor es la aplicación viva y funcionando. Es el **Food Truck ya fabricado y abierto al público**. Si tienes el manual de la franquicia (la *Imagen*), puedes decidir abrir 10 Food Trucks (10 *Contenedores*) exactamente iguales en distintas calles de tu ciudad. Todos trabajarán de forma independiente, pero todos habrán salido del mismo molde.

2.3. Docker Hub: El supermercado de ingredientes

Afortunadamente, no tienes que crear todos tus manuales desde cero. Existe un lugar en internet llamado **Docker Hub**. Es como el catálogo mundial de franquicias. ¿Necesitas una base de datos de PostgreSQL? Vas a Docker Hub y te descargas el manual oficial gratis. ¿Necesitas un servidor web? Descargas el manual de Nginx. La inmensa mayoría de las herramientas que usarás como desarrollador ya tienen su propia imagen oficial lista para descargar y usar.

2.4. Cliente vs. Servidor: Quién manda y quién obedece

Docker funciona bajo una estructura de **Cliente** / **Servidor**, lo cual suele confundir a los principiantes que usan Windows, ya que todo ocurre dentro del mismo ordenador.

Imagina que tu ordenador es la ciudad entera.

- **El Cliente (Tú):** Eres el inversor. Te sientas en tu oficina (la terminal de comandos o PowerShell) y das órdenes tecleando cosas como *“¡Constrúyeme un Food Truck de hamburguesas!”*.
- **El Servidor / Docker Daemon (El Capataz):** Es un programa silencioso que corre en el fondo de tu Windows (Docker Desktop). Él es quien escucha tus órdenes, va a Docker Hub a por el manual, construye el Food Truck y lo pone a funcionar. Tú nunca construyes el contenedor directamente; se lo pides al Capataz.

3. Preparando el terreno: Instalación en Windows 11

Ya conocemos la teoría y tenemos los planos de nuestra franquicia de Food Trucks. Ahora necesitamos que la "empresa constructora" (Docker) se instale en nuestra ciudad (Windows 11) para empezar a trabajar.

De forma técnica, nuestro sistema operativo Windows 11 será el equipo **host**, mientras que mediante la instalación de Docker conseguiremos tener varios sistemas **hosted** en un mismo equipo.

3.1. ¿Qué es WSL2 y por qué Windows necesita un "corazón de Linux"?

Docker fue creado originalmente para funcionar en sistemas operativos Linux. Usando nuestra analogía: El capataz de la obra de Docker solo habla el idioma "Linux". Si intentamos que trabaje directamente sobre las calles de Windows, no va a entender cómo utilizar los recursos (la memoria, el procesador, las redes).

Para solucionar esto, Microsoft creó **WSL2** (Windows Subsystem for Linux). Piensa en WSL2 como un barrio especial dentro de tu ciudad de Windows donde las reglas y el idioma son 100 % Linux. Al instalar WSL2, le estamos dando al capataz de Docker el terreno perfecto para que construya los Food Trucks a la máxima velocidad y sin problemas de compatibilidad.

3.2. Instalación paso a paso de Docker Desktop

Vamos a preparar el terreno y a contratar a nuestro capataz.
Sigue estos pasos al pie de la letra:

1. **Prepara el terreno (Instala WSL2):** Abre tu menú de inicio, busca *PowerShell*, haz clic derecho y selecciona *"Ejecutar como administrador"*. Ejecutaremos:

```
wsl --install
```

Terminal 1: Instalación del Subsistema Linux en Windows

Nota: Si al ejecutarlo te dice que ya está instalado o te muestra el menú de ayuda, significa que tu Windows 11 ya venía preparado de fábrica..

2. **Reinicia tu ordenador:** Esto es obligatorio para que el nuevo "barrio Linux" se construya correctamente en tu sistema.

   Tras este paso, puede que WSL haya instalado y descargado a la misma vez una distribución de Linux, o puede que no. Lo comprobaremos en el siguiente paso.

3. **Comprobar WSL:** Sería interesante, antes de seguir con otras cosas, ver qué ha instalado realmente. Volveremos a abrir *PowerShell* en modo Administrador, y se comprueba:

```
wsl --list --all
# Te debe indicar la distribución que se instaló, como las
   siguientes líneas:
Distribuciones de subsistema de Windows para Linux:
Ubuntu (Predeterminado)
```

Terminal 2: Comprobación del Subsistema Linux en Windows

Si no aparece ninguna distribución predeterminada, es recomendable descargar e instalar alguna de ellas.

En estos casos, lo más normal es usar Ubuntu:

```
wsl --install -d Ubuntu
```

Terminal 3: Instalación de Ubuntu en el WSL de Windows

Cuando ejecutamos esta instrucción se realizará la instalación de un sistema base sobre el que trabajar.

Te pedirá un usuario y una contraseña, que será el de acceso como **root** a esta distribución de Linux.

4. **Descarga la empresa constructora:** Ve a la página oficial de Docker (https://www.docker.com/products/docker-desktop/) y descarga el instalador para Windows.

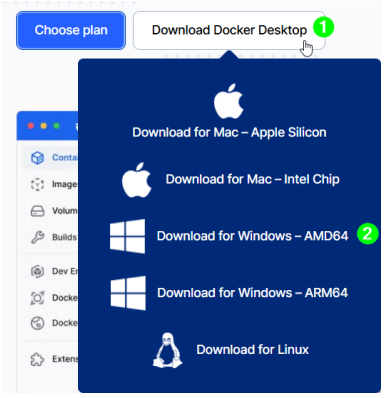

Figura 1: Descarga del Docker Desktop para Windows

5. **Ejecuta el instalador:** Se necesitarán privilegios de administrador para poder realizar la instalación de Docker Desktop.

   Durante la instalación, te aparecerá una ventana con opciones a tener en cuenta.

   Es vital asegurarse de que la opción ***"Use WSL 2 instead of Hyper-V"*** esté marcada.

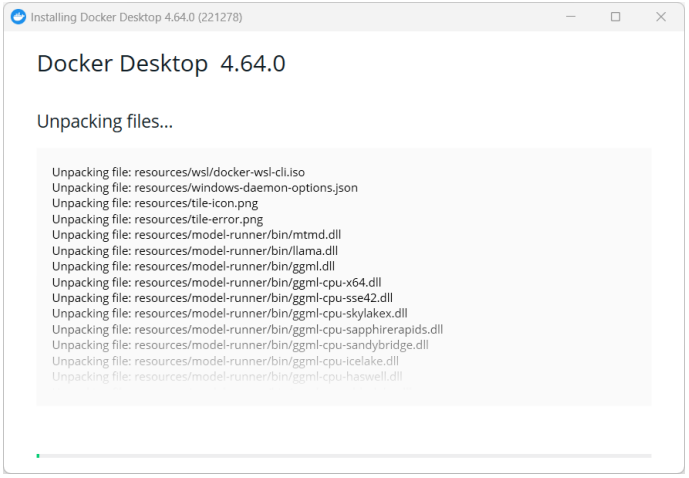

Figura 2: Proceso de instalación de Docker Desktop en Windows

6. **Finaliza y abre Docker:** Una vez termine, busca *Docker Desktop* en tu menú de inicio y ábrelo. Ten paciencia, que le cuesta arrancar.

Figura 3: Cuando arranque, aparecerá esta imagen

Tras arrancar, verás el icono de una ballena en la barra de tareas (junto al reloj). Presionando sobre ella con el botón derecho, te aparecerá el siguiente menú contextual:

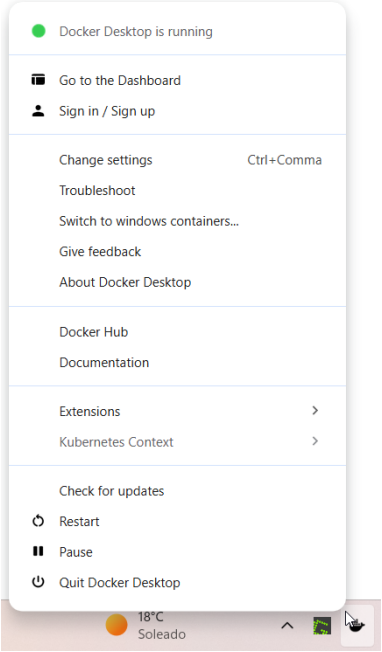

Figura 4: Menú contextual sobre el icono de la barra de tareas

3.3. Tour por la interfaz gráfica de Docker Desktop

Aunque a partir de ahora daremos las órdenes escribiendo comandos (porque es como los profesionales construyen infraestructuras a prueba de fallos), Docker Desktop nos ofrece una ventana visual para ver qué está pasando.

Al abrir la aplicación, fíjate en el menú lateral izquierdo.

Para entenderlo de un vistazo rápido, volvamos a nuestra franquicia de *Food Trucks*:

- **Containers (Los Food Trucks):** Identificados en la imagen con 1

  Es tu flota actual de camiones. Aquí ves cuáles de los *Food Trucks* están cocinando, cuáles están aparcados y puedes asomarte a su interior.

- **Images (Los Manuales):** dentificados en la imagen con 2

  Es la estantería con un catálogo de planos y recetas descargados, listos para ser usados en la construcción de nuevos camiones.

- **Volumes (Las Cajas Fuertes):** dentificados en la imagen con 3

  Son tus almacenes externos independientes, donde guardas los **datos críticos** a salvo de posibles demoliciones.

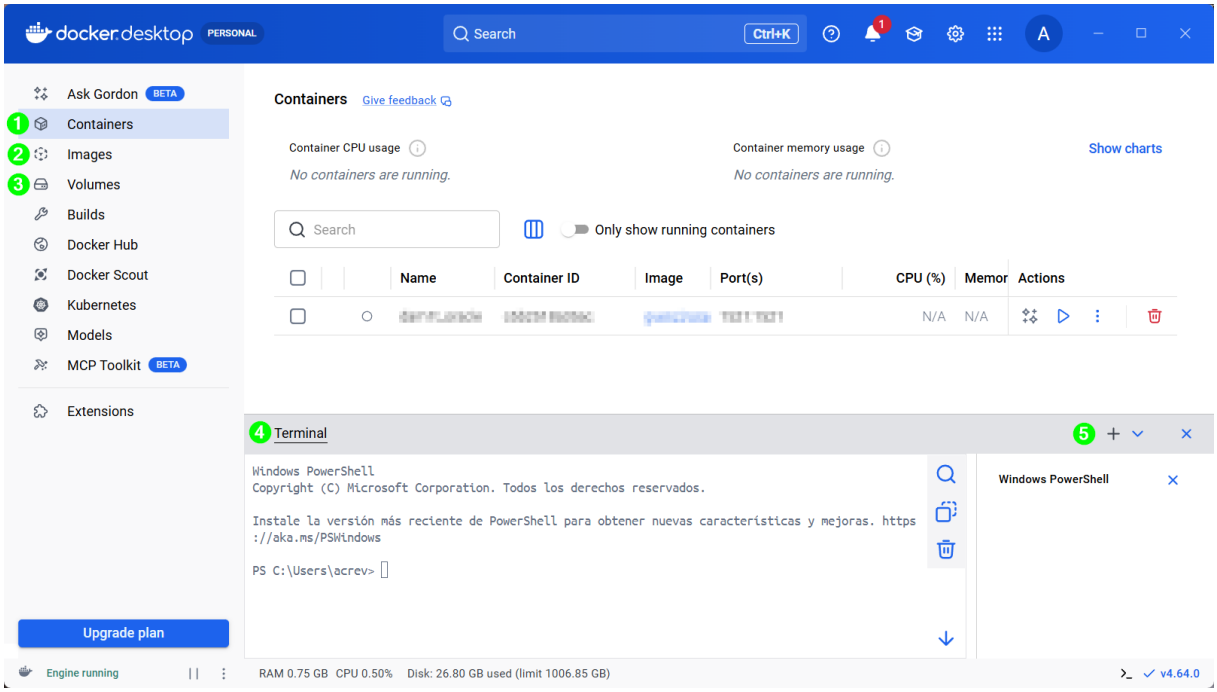

Figura 5: Ventana principal de Docker Desktop

En la imagen también nos encontramos un terminal, dentificado en la imagen con 4 y 5, por lo general de **PowerShell**, donde podremos escribir los comandos para manejar Docker.

Profundizando: La Visión Técnica

Una vez revisada la parte visual y las analogías con las ideas que habíamos desarrollado de la empresa de *Food Trucks*, vamos a realizar una definición más técnica de los conceptos básicos necesarios para poder trabajar con Docker:

- **Images**

  Una **Imagen** de Docker es una plantilla **inmutable y de solo lectura** compuesta por múltiples capas superpuestas de sistemas de archivos (basadas en el sistema de archivos de unión o *UnionFS*), que contiene el código fuente, las bibliotecas binarias, las dependencias, las herramientas y las variables de entorno necesarias para ejecutar una aplicación.

  Se construye de forma automatizada a partir de un archivo de manifiesto declarativo (`Dockerfile`) y sirve como el *artifact* estandarizado de empaquetado que **garantiza la inmutabilidad y portabilidad del software entre diferentes infraestructuras**, siendo almacenada, versionada mediante etiquetas (*tags*) y distribuida a través de registros de contenedores.

- **Containers**

  Un **Contenedor** es una **instancia ejecutable, efímera y aislada de una imagen**, que opera como un proceso nativo dentro del kernel del sistema operativo anfitrión, utilizando *namespaces* para el aislamiento lógico del espacio de trabajo (redes, procesos, puntos de montaje) y *cgroups* para la limitación estricta de recursos de hardware (como son la CPU, memoria,..., entre otros).

  Al iniciar un contenedor, Docker aprovisiona una capa superior de lectura y escritura (*Copy-on-Write*) sobre las capas inmutables de la imagen subyacente, lo que permite

que la aplicación en ejecución modifique su estado interno de forma aislada sin alterar la imagen base ni afectar a otros procesos del host.

Es necesario tener en cuenta que, cuando un *Container* se detiene, todas las modificaciones realizadas durante su ejecución se perderán, a menos que se haya configurado de alguna forma para mantener dichas modificaciones entre sesión y sesión de trabajo. Esa herramienta son los volúmenes.

- **Volumes**

  Un **Volumen** es el mecanismo nativo y de alto rendimiento, proporcionado por Docker para la persistencia de datos y el intercambio de estado entre contenedores. Consiste en un sistema de archivos almacenado en una ruta específica del disco del host (típicamente `/var/lib/docker/volumes/` en entornos Linux) que es administrado de forma exclusiva por el *Docker Daemon*.

  A diferencia de la capa efímera de lectura y escritura del contenedor, los volúmenes están completamente desacoplados del ciclo de vida de la instancia que los utiliza, omiten la sobrecarga del sistema de archivos de unión (*UnionFS*) para mejorar la latencia de entrada/salida (I/O) y garantizan que la información crítica sobreviva a los procesos de Parada/Inicio/Reinicio del contenedor al que están asignados, así como también sobreviven a la destrucción del contenedor.

Administración de los componentes de Docker

Vamos a ver lo que realmente ofrece cada una de las pestañas a nivel de administración en cada uno de los apartados.

1. Imágenes *(Images)*: Gestión de Artefactos y Seguridad

En este apartado se gestionan los binarios y artefactos locales que has descargado de los registros (como Docker Hub) o que se han construido *(build)* (tú mismo o cualquier otra persona, pero no son oficiales) a partir de un Dockerfile.

En la ventana de administración de Imágenes, podemos realizar lo siguiente:

- **Control de Almacenamiento**

  Las imágenes están formadas por múltiples capas (layers) de solo lectura, que ocupan espacio en el disco de tu Windows (dentro del disco virtual de WSL2).

  Esta vista te detalla el peso exacto de cada imagen y su versión o etiqueta (`tag`, como `latest`, `alpine` o `1.2.0`).

- **Imágenes Huérfanas (Dangling Images)**

  Permite filtrar las imágenes marcadas como *Unused*. Suelen ser versiones antiguas o capas intermedias que ya no están asociadas a ningún contenedor activo ni tienen una etiqueta válida. Eliminarlas desde aquí (equivalente a `docker image prune`) es vital para evitar que el disco duro se llene con el tiempo.

- **Análisis de Vulnerabilidades**

  En las versiones modernas, Docker se integra con herramientas (como Docker Scout) para inspeccionar los paquetes instalados en las capas de la imagen. Te alertará si la imagen base de tu servidor web contiene vulnerabilidades de seguridad conocidas (CVEs) antes siquiera de que la ejecutes.

Como ejemplo práctico, si nos fijamos en la siguiente imagen, podremos diferenciar las diferentes partes indicadas más arriba:

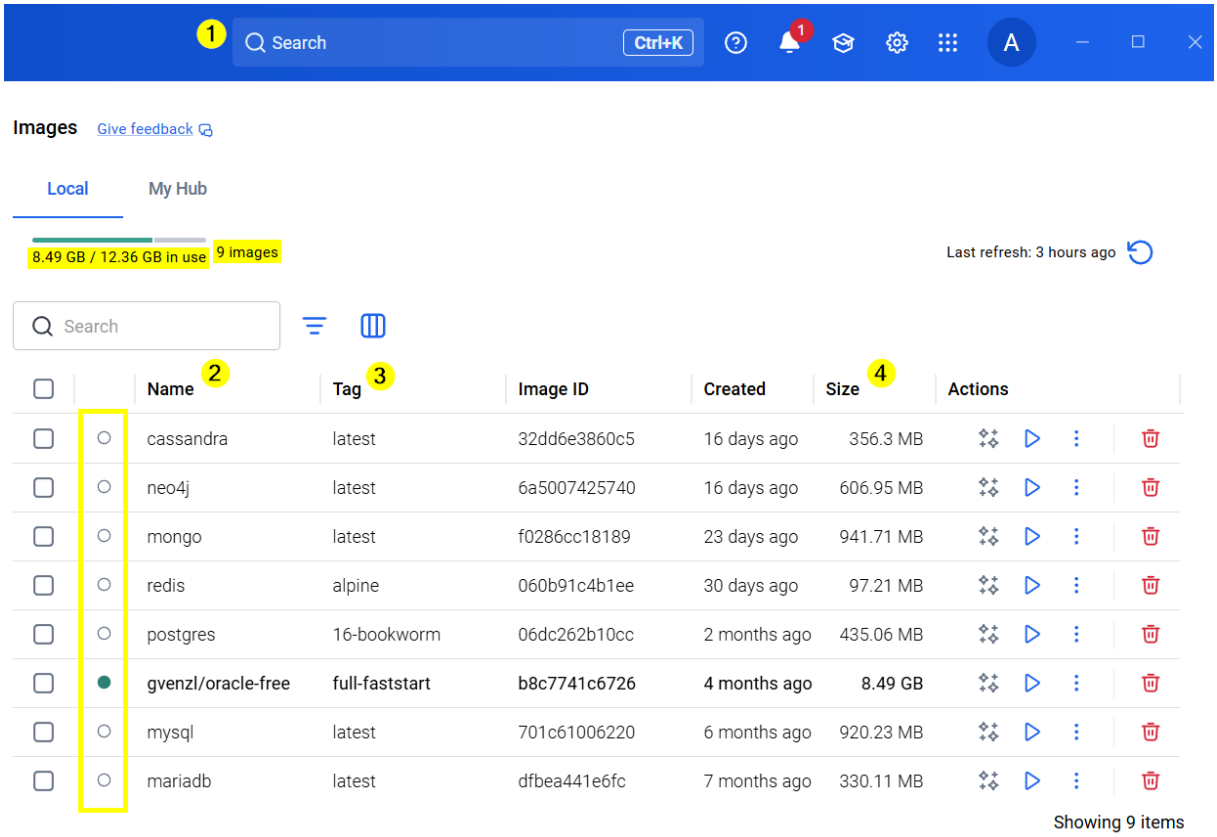

Figura 6: Ejemplo de ventana de Administración de Imágenes

Analicemos las diferentes partes de la ventana de Administración de Imágenes en Docker Desktop:

- En el punto 1 nos encontramos un buscador de imágenes, donde podremos encontrar lo que necesitamos tan solo escribiendo el nombre.

  Si no tenemos claro el nombre de la *Image* que se está buscando, siempre podemos hacerlo desde **Docker Hub**, donde las imágenes más comunes están ordenadas por categoría.

- Un poco más abajo nos encontramos el espacio ocupado por todas las Images descargadas (12,36 GB en esa imagen) y las que están realmente en uso (8,49 GB).

  Para saber cuáles están en uso y cuáles no, debemos bajar al círculo que aparece antes del nombre de la *Image*. Si observas bien en la imagen de ejemplo, en el recuadro amarillo se puede ver que cada *Image* tiene delante un círculo, que puede tener dos estados:

  - Si el círculo está vacío significa que esa *Image* no tiene un *Container* que lo esté usando, mientras que,
  - Si el círculo está en verde sí que existe un *Container que lo utilice*.

- Si miramos ahora en cada una de las filas, podemos identificar los valores para el nombre 2, su versión o *Tag* 3 así como su fecha de descarga y el tamaño que ocupa en el almacenamiento del *equipo host* 4

- **Ten en cuenta lo siguiente:** para eliminar una *Image*, es necesario que no tenga ningún *Container* asociado al mismo.

2. Contenedores *(Containers)* y su Gestión del Ciclo de Vida (y Debugging)

Esta parte es el monitor de trabajos o servicios existentes, ya sean parados o en ejecución, en tiempo real. No solo lista los contenedores existentes, sino que expone sus interfaces y procesos internos:

- **Control de estado**

  Permite enviar señales del sistema del host al contenedor (como `SIGTERM` para detener suavemente o `SIGKILL` para forzar el cierre) mediante botones gráficos.

- **Mapeo de Puertos Activo**

  Muestra los puertos expuestos de un contenedor en concreto y cómo están mapeados al host de Windows.

  Por ejemplo, con el mapeo `8080:80`) en un *container* de un Servidor Web, al escribir el hipervínculo `http://localhost:8080` en un navegador, tu equipo `host` verá el puerto y realizará un mapeo hacia el *container* correspondiente y hacia el puerto adecuado (el `80` en el container destino), facilitando la prueba inmediata de una página Web o de Web Services desplegados.

- **Inspección de Logs (Standard Output/Error)**

  Al hacer clic en el nombre de un contenedor, accedes a su pestaña de *Logs*. Esto captura en tiempo real la salida estándar (`stdout`) y de error (`stderr`) del proceso principal del contenedor. Es absolutamente fundamental para depurar por qué una base de datos o un servidor ha fallado al intentar arrancar.

- **Terminal Interactiva (Exec)**

  Desde la pestaña *Exec*, Docker Desktop inyecta una consola conectada directamente al interior del contenedor. Es equivalente a `docker exec -it <contenedor>sh` desde *PowerShell*. Esto nos permitirá navegar por su sistema de archivos Linux virtualizado en vivo para comprobar rutas, permisos o instalar utilidades temporales.

- **Estadísticas de Recursos (Stats)**

  La pestaña *Stats* muestra gráficas de telemetría en vivo del consumo exacto de CPU, memoria RAM, lectura/escritura en disco e I/O de red aislado para ese contenedor específico.

Figura 7: Opciones de administración de un *Container* existente

En la imagen anterior podemos encontrar un ejemplo práctico de todo lo visto en este punto, sobre un *Container* existente.

3. Volúmenes *(Volumes)* que ofrecen Persistencia

Si no lo evitamos, cada vez que se pare y arranque de nuevo un *Container*, todas las modificaciones que se hubieran realizado sobre dicha instancia desaparecerán.

Esto se debe a que, por defecto, todo se ejecuta sobre una **capa efímera** de lectura/escritura *(Copy-on-Write)* del *Container*.

Se usan los Volúmenes *(Volumes)* para facilitar la gestión del sistema de archivos persistentes, que escaparán del manejo de la capa efímera *(Copy-on-Write)* indicada antes.

Estas son las características de los Volúmenes:

- **Ciclo de vida independiente**

  Los *Volumes* son gestionados directamente por Docker Daemon (ubicados por lo general en el entorno de Linux subyacente `/var/lib/docker/volumes/`) y **tienen una vida útil totalmente separada de los contenedores que los utilizan**.

- **Explorador de Datos integrado**

  Históricamente, ver qué había dentro de un volumen en Windows era complejo.

  Ahora, la interfaz gráfica te permite hacer clic en un volumen y explorar un árbol de directorios con los archivos reales de tu base de datos o aplicación, sin necesidad de montar el volumen en un contenedor temporal solo para poder leerlo.

- **Gestión de huérfanos y Backups**

  Es muy común borrar un contenedor desde la terminal y olvidarse de borrar su volumen asociado. Esta pestaña te permite identificar rápidamente volúmenes que ya no están unidos a ningún *Container* y pueden borrarse.

  Además, ofrece opciones nativas para exportar el contenido de un volumen a un archivo comprimido `.tar.gz` directamente a tu Escritorio de Windows, facilitando las copias de seguridad.

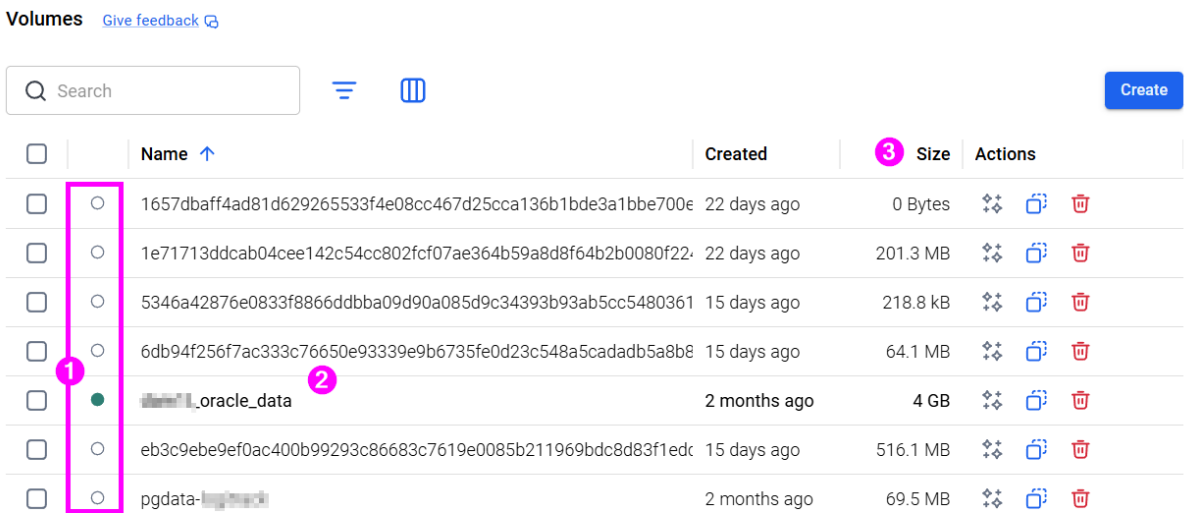

Figura 8: Ventana de administración de *Volumes* en Doker Desktop

En esta imagen podemos ver en 1 qué Volumenes están en uso y cuáles no, así como el nombre en 2 y el tamaño que ocupan en 3.

Aunque no es obligatorio, es recomendable dar a los Volúmenes nombres descriptivos para saber qué tienen. Si no se hace, se les dará un UUID como nombre.

3.4. Configuración básica de recursos (RAM y CPU)

En las versiones antiguas de Docker, tenías que decirle manualmente al capataz cuánta memoria RAM podía gastar.

Hoy en día, gracias a WSL2, **Docker es inteligente y dinámico**. Toma prestada la RAM de tu Windows solo cuando el *Container* lo necesita (los Food Trucks tienen muchos clientes), y la devuelve cuando terminan. No necesitas configurar nada para empezar.

Sin embargo, si alguna vez notas que tu Windows 11 va lento porque Docker está consumiendo demasiado, puedes ponerle un límite. Solo tienes que ir al icono de Ajustes (arriba a la derecha en Docker Desktop), y en la sección *Resources*, configurar un máximo de memoria o procesadores.

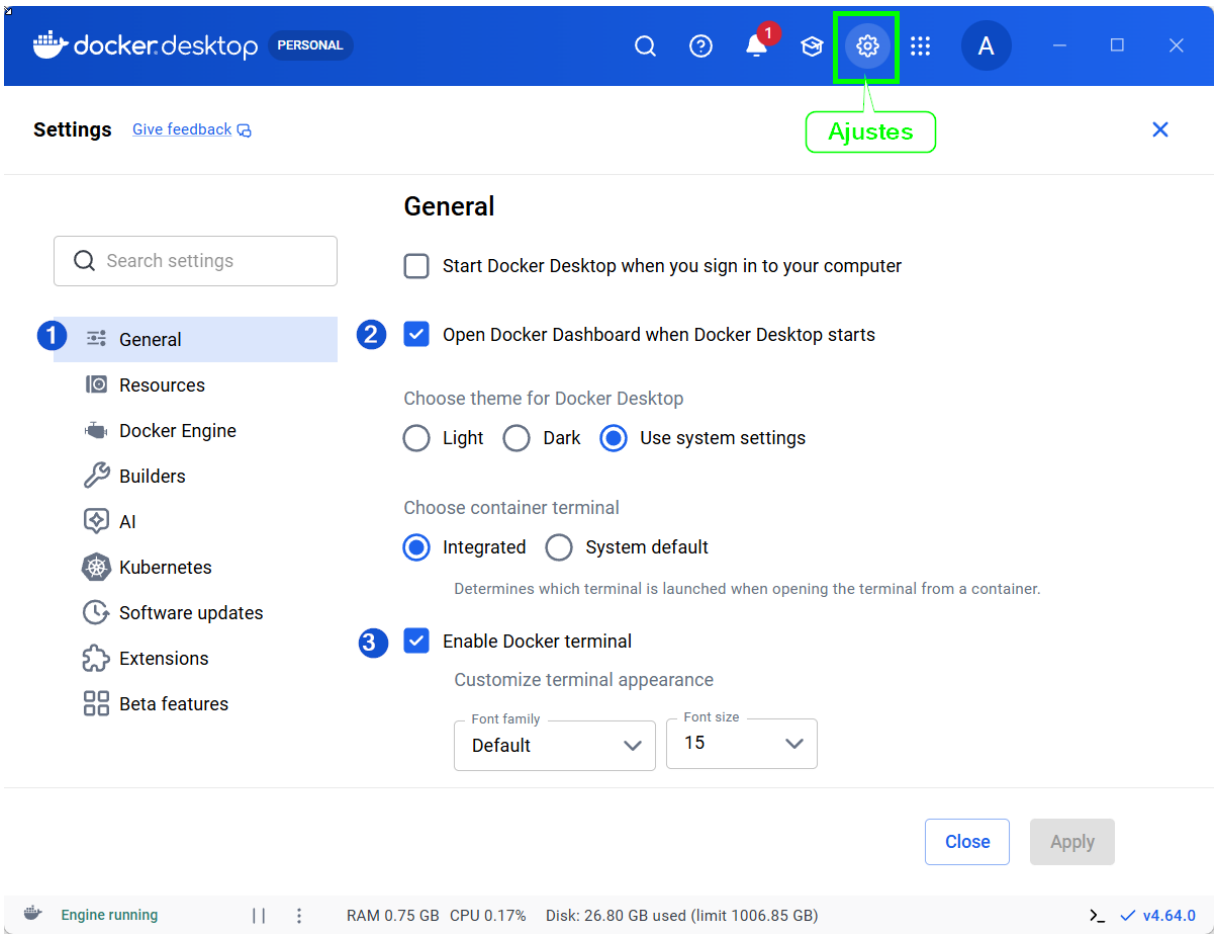

Figura 9: Configuración básica de Ajustes / General

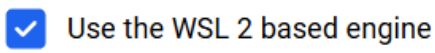

Figura 10: Recomendación: Integrar Docker Desktop con WSL2

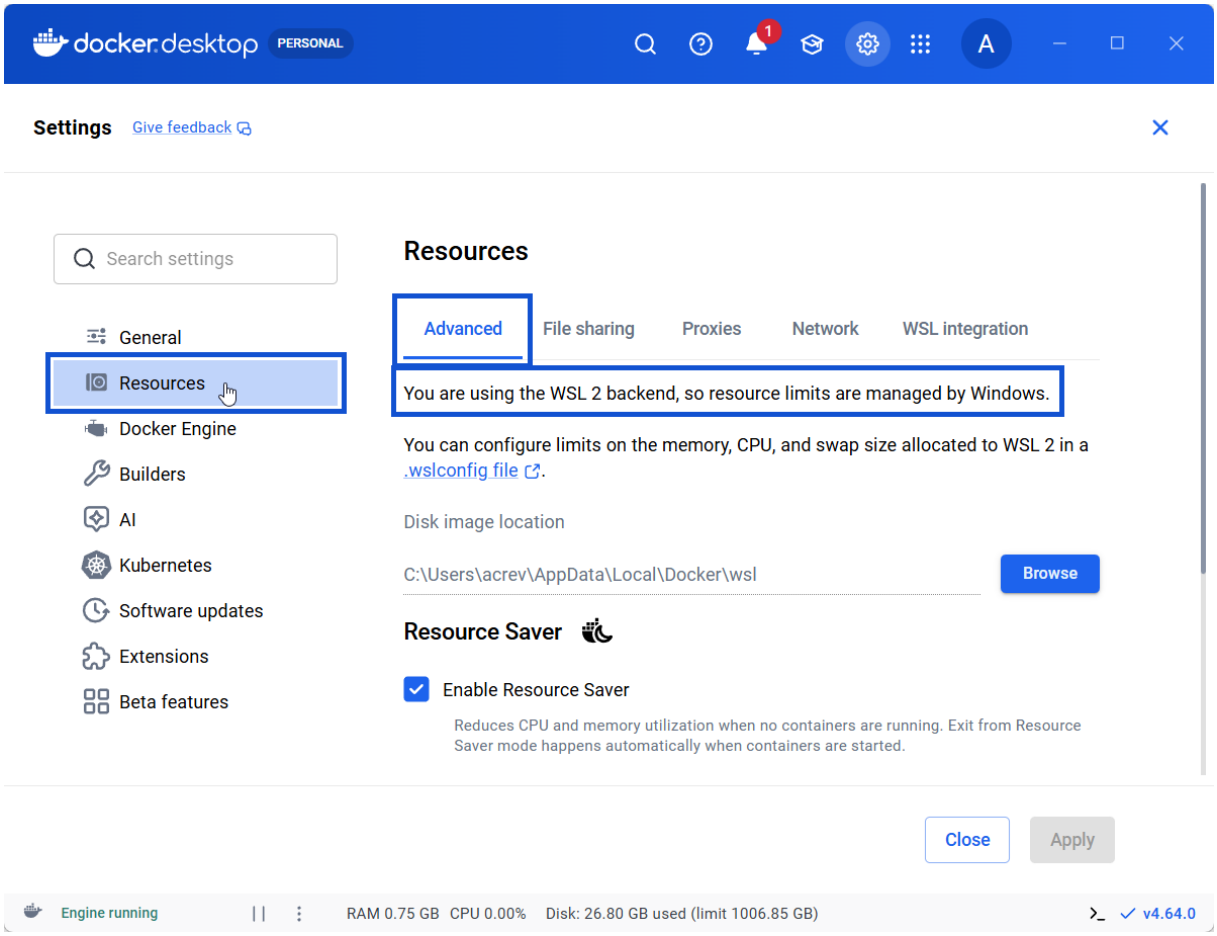

Figura 11: En vez de configurar aquí los recursos, se hará configurando WSL2

4. Primeros pasos: Hola Mundo

Es hora de mancharse las manos. Ya tenemos el terreno preparado, el capataz de Docker está esperando órdenes en Windows 11, y nosotros estamos listos en nuestra oficina (la terminal).

Vamos a pedirle que abra nuestro primer local de prueba.

4.1. Tu primera orden en la terminal (PowerShell)

En Windows 11, tu centro de mando principal será **PowerShell**. Abre el menú de inicio, escribe *PowerShell* y ábrelo. Verás una pantalla azul o negra con letras blancas. No te asustes, aquí es donde ocurre la magia.

Para comprobar que todo funciona perfectamente y que el capataz nos escucha, vamos a teclear nuestra primera orden y pulsar Enter:

```
docker run hello-world
```

Terminal 4: Tu primer contenedor

Si todo ha ido bien, verás que aparecen varias líneas de texto en tu pantalla que empiezan diciendo: *"Hello from Docker!"*. ¡Felicidades! Acabas de ejecutar tu primer contenedor. Pero, ¿qué ha pasado exactamente?

4.2. Entendiendo qué ha pasado "por detrás"

Ese simple comando de tres palabras (`docker run hello-world`) ha desencadenado una serie de acciones complejas que el capataz ha resuelto en segundos. Siguiendo nuestra analogía de la franquicia, esto es lo que ha ocurrido paso a paso:

1. **La orden:** Tú (el cliente) has tecleado `docker run`, que significa *"Capataz, constrúyeme y pon a funcionar un Food Truck de la franquicia hello-world"*.
2. **La búsqueda local:** El capataz busca en su biblioteca de Windows si ya tiene el manual (la *Imagen*) de "hello-world". Como es la primera vez, se da cuenta de que no lo tiene.
3. **La descarga (Docker Hub):** Sin que tú tengas que hacer nada, el capataz se conecta a internet, va al catálogo mundial (Docker Hub), busca el manual oficial de "hello-world" y lo descarga en tu ordenador.
4. **La construcción:** Con el manual en la mano, construye un nuevo Food Truck (el *Contenedor*).
5. **La ejecución:** Abre el Food Truck. Este contenedor en particular está programado para hacer una sola cosa: imprimir un mensaje de saludo en tu pantalla.
6. **El cierre:** Una vez que imprime el mensaje, el Food Truck termina su trabajo y se cierra automáticamente.

```
PS C:\> docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
Digest: sha256:85404
   b3c53951c3ff5d40de0972b1bb21fafa2e8daa235355baf44f33db9dbdd
```

```
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working
   correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker
    Hub. (amd64)
 3. The Docker daemon created a new container from that image which
    runs the process that produces the output you are reading.
 4. The Docker daemon streamed that output to the Docker client, which
     sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
    docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
    https://hub.docker.com/

For more examples and ideas, visit:
    https://docs.docker.com/get-started/
```

Terminal 5: Salida de ejecutar en PowerShell: docker run hello-world

4.3. El ciclo de vida: Crear, Iniciar, Parar y Borrar

El contenedor de “hello-world” es especial porque se apaga solo tras hacer su tarea. Pero en la vida real, los Food Trucks (como un servidor web o una base de datos) se abren por la mañana y se quedan trabajando todo el día hasta que tú decides cerrarlos.

Para gestionar esto, necesitamos conocer los comandos básicos del **ciclo de vida**.

Vamos a usar una imagen real de un servidor web llamado `nginx` para practicar.

4.3.1. Crear y dejar trabajando en segundo plano (`-d`)

Si ejecutas un servidor y te quedas mirando la consola, la pantalla se quedará bloqueada mostrándote lo que hace el servidor y no podrás teclear nada más. Es como si te quedaras dentro del Food Truck vigilando al cocinero en lugar de irte a dirigir la empresa.

Para decirle al capataz que abra el local y te deje volver a tus asuntos en la oficina, usamos el parámetro **-d** (que significa *detached* o desconectado).

```
    docker run -d nginx
```

Terminal 6: Abrir un servidor web en segundo plano

El capataz descargará la imagen de `nginx`, construirá el local, lo dejará funcionando y te devolverá una cadena larga de letras y números por pantalla. Ese código es el número de matrícula o identificación de tu nuevo Food Truck.

4.3.2. Ver los locales abiertos (`docker ps`)

Si quieres saber cuántos Food Trucks están trabajando ahora mismo en tu ciudad, usas el comando de listar procesos (`ps`):

```
docker ps
```

Terminal 7: Listar contenedores activos

```
PS C:\Users\[illegible]> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS     NAMES
292c9dbe3769   nginx     "/docker-entrypoint.…"   9 seconds ago   Up 8 seconds   80/tcp    charming_pasteur
```

Figura 12: Resultado de ejecución de: docker ps

Fíjate en la última columna de tu consola llamada *NAMES*. Como no le dimos un nombre a nuestro camión al crearlo, el capataz le ha asignado uno aleatorio y divertido compuesto por un adjetivo y el nombre de un científico famoso (por ejemplo, *crazy_einstein* o *sleepy_turing*). Puedes usar ese nombre, o los primeros 3 o 4 números de la columna *CONTAINER ID*, para darle órdenes a ese camión en concreto.

4.3.3. Cerrar el local por hoy (`docker stop`)

Cuando termina el día y quieres que el camión deje de gastar recursos (la memoria RAM y el procesador de tu Windows), debes detenerlo. Sustituye “nombre_del_camion” por el nombre o el ID que te haya salido a ti en el paso anterior.

```
docker stop nombre_del_camion
```

Terminal 8: Detener un contenedor

Si ahora vuelves a escribir `docker ps`, verás que la lista está vacía. ¿Significa que el camión se ha destruido? ¡No! Solo está aparcado y con la persiana bajada. Para ver todos los camiones que posees, tanto los abiertos como los cerrados, usa el parámetro `-a` (*all*):

```
docker ps -a
```

Terminal 9: Ver todos los contenedores (activos e inactivos)

```
PS C:\Users\[illegible]> docker ps --all
CONTAINER ID   IMAGE                          COMMAND                  CREATED             STATUS                        PORTS     NAMES
292c9dbe3769   nginx                          "/docker-entrypoint.…"   About a minute ago  Up About a minute             80/tcp    charming_pasteur
1c9e766b7939   hello-world                    "/hello"                 18 minutes ago      Exited (0) 18 minutes ago               gifted_jackson
6bc4d377dfbf   hello-world                    "/hello"                 20 minutes ago      Exited (0) 18 minutes ago               wonderful_montalcini
c6b2518a35ac   [illegible] full-faststart     "container-entrypoin…"   2 months ago        Exited (137) 29 minutes ago             [illegible]
```

Figura 13: Resultado de ejecución de: docker ps —all

4.3.4. Volver a abrir al día siguiente (`docker start`)

No necesitas volver a usar `docker run` (recuerda que eso construiría un camión nuevo desde cero y tendrías dos camiones). Solo necesitas despertar al que ya tienes aparcado:

```
docker start nombre_del_camion
```

Terminal 10: Iniciar un contenedor detenido

4.3.5. Demoler el local (`docker rm`)

Si ya no necesitas ese servidor web y quieres deshacerte de él para recuperar espacio en tu disco duro, primero debes detenerlo (usando `docker stop`) y luego ordenar su demolición (`rm` de *remove*):

```
docker rm nombre_del_camion
```

Terminal 11: Eliminar un contenedor definitivamente

¡Cuidado!
Al hacer `docker rm`, el contenedor se destruye por completo.
Si tenías datos guardados dentro (como los clientes registrados en una base de datos o fotos subidas a tu web) y no usaste una "Caja Fuerte" (Volumen), esa información se perderá para siempre y no podrá recuperarse. Veremos cómo evitar este desastre más tarde.

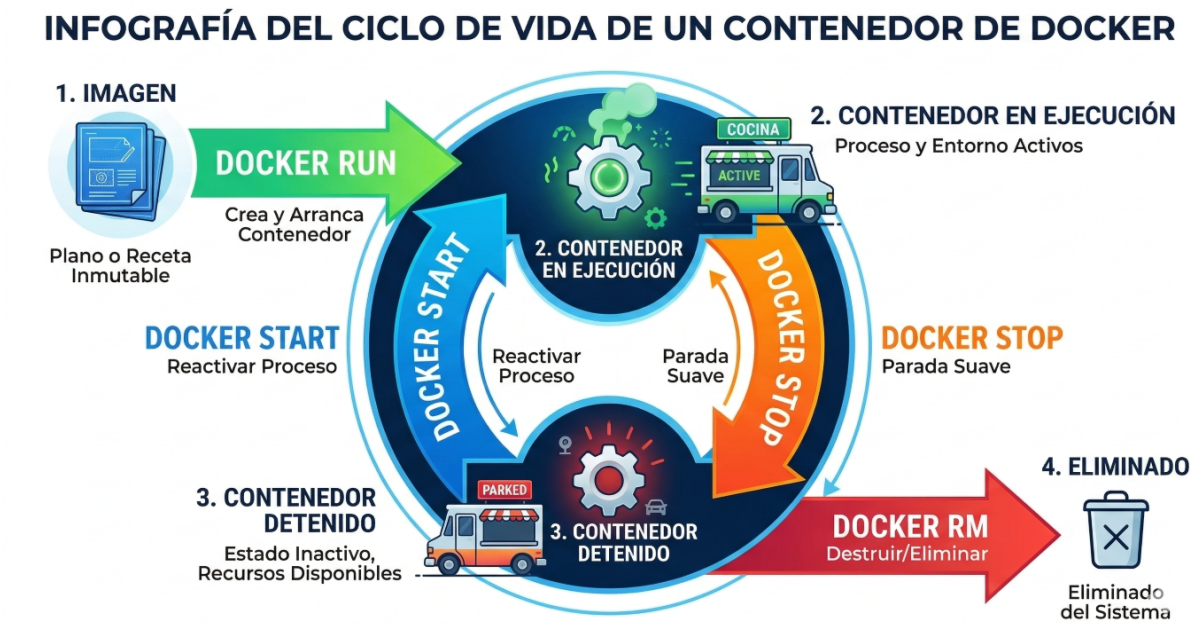

Figura 14: Ciclo de vida de un *Container*

5. Redes para quien no sabe de Redes

En el apartado anterior, conseguimos que nuestro capataz abriera un Food Truck (ejecutamos un contenedor con `nginx`, que es un servidor web muy famoso).

Sin embargo, si ahora mismo abres tu navegador de internet en Windows (Chrome, Edge, Firefox) y tratas de ver esa página web, no encontrarás nada. Es como si el camión no existiera.

¿Por qué ocurre esto si el capataz nos confirmó que está funcionando?

La respuesta está en el aislamiento.

5.1. El concepto de aislamiento: Tu contenedor es una caja fuerte

Por defecto, cuando Docker construye un contenedor, lo hace pensando en la **seguridad extrema**.

Imagina que el Food Truck que acaba de construir el capataz es, literalmente, una caja blindada de titanio sin puertas ni ventanas. El cocinero está dentro, la cocina está encendida, las hamburguesas se están haciendo y todo funciona a la perfección. Pero si tú, como cliente, te acercas desde la calle, no tienes por dónde pedir la comida ni cómo ver hacia adentro.

Este aislamiento es **una de las grandes ventajas de Docker**: un contenedor hackeado o defectuoso no puede afectar a tu Windows 11 porque está encerrado. Pero claro, si es un servidor web, necesitamos que la gente pueda entrar a ver la página. Necesitamos hacer un agujero controlado en esa caja blindada.

5.2. Conceptos previos: ¿Qué es una IP y qué es un Puerto?

Para entender cómo conectar Windows con el contenedor, primero tenemos que entender cómo se comunican los ordenadores en general. Olvídate de Docker por un minuto.

5.2.1. La Dirección IP (La calle y el número)

Cada dispositivo conectado a una red (tu ordenador, tu móvil, tu Smart TV) tiene una **Dirección IP**. Es como la dirección postal de un edificio. Por ejemplo, la IP de tu ordenador podría ser `192.168.1.50`. Si un paquete de datos quiere llegar a tu PC, el router de tu casa lee esa dirección y se lo entrega.

5.2.2. El Puerto (La puerta o ventanilla específica)

Pero una vez que el paquete llega al edificio (tu PC), ¿por qué puerta debe entrar? Un ordenador hace muchas cosas a la vez: navega por internet, recibe correos, descarga archivos. Para no mezclar las cosas, el ordenador tiene **65.535 "puertas"numeradas**, llamadas Puertos.

Siguiendo la analogía:

- **El edificio (IP):** Es tu ordenador.
- **Puerta 80 (Puerto 80):** Es la puerta principal de cristal por donde entran los clientes (Navegación Web HTTP).
- **Puerta 443 (Puerto 443):** Es la puerta blindada para clientes VIP (Navegación Web Segura HTTPS).
- **Puerta 3306 (Puerto 3306):** Es la puerta trasera por donde los proveedores entregan la mercancía (Base de Datos).

5.3. El problema de los choques en Windows

Imagina que quieres instalar tres bases de datos distintas en tu Windows para probarlas. Si las instalas de forma normal (sin Docker), las tres van a intentar usar la misma puerta trasera (el puerto 3306). La primera que llegue la bloqueará, y las otras dos te darán un error diciendo: *"¡El puerto ya está en uso!"*.

Aquí es donde Docker brilla con luz propia.

Como cada contenedor es una caja blindada aislada, cada contenedor tiene **sus propios 65.535 puertos internos**. Puedes tener cien bases de datos funcionando en Docker, y todas estarán usando su propio puerto 3306 *dentro* de su propio contenedor, sin pelearse entre ellas.

5.4. El comando `-p` (Mapeo de puertos): Construyendo el puente

Ahora volvamos a nuestro servidor web Nginx. Nginx es un servidor web, por lo que su cocinero interno siempre está esperando clientes asomado a la ventanilla **80** dentro de su caja blindada.

Para que tú desde tu navegador en Windows puedas hacerle un pedido, tenemos que decirle al capataz que construya un tubo o puente que conecte una puerta libre de tu Windows, directamente con la ventanilla 80 del Food Truck. A esto se le llama **Mapeo de Puertos** o *Port Forwarding*.

Se utiliza el parámetro `-p` (minúscula) seguido de dos números separados por dos puntos (`:`):

```
docker run -p PuertoEnWindows:PuertoEnDocker nombre_de_la_imagen
```

Terminal 12: Sintaxis del mapeo de puertos

Regla de Oro para principiantes: En Docker, lo que está a la **Izquierda** de los dos puntos es TU MÁQUINA (Windows). Lo que está a la **Derecha** es el CONTENEDOR.

Vamos a abrir un nuevo Food Truck de Nginx (recuerda usar `-d` para que trabaje de fondo) y vamos a decirle: *Çapataz, conecta la puerta **8080** de mi Windows a la puerta **80** del camión"*.

```
docker run -d -p 8080:80 nginx
```

Terminal 13: Ejecutar servidor web con puerto mapeado

5.5. ¿Qué es `localhost`? Accediendo a tu aplicación

Ya tenemos el puente construido. Si alguien llama a la puerta 8080 de tu Windows, el capataz lo cogerá en volandas por el tubo secreto y lo dejará en la ventanilla 80 del contenedor.

Para probarlo, abre tu navegador web (Chrome, Firefox, etc.). En la barra de direcciones superior (donde sueles escribir www.google.com), vas a escribir la dirección de tu propia máquina.

En informática, la palabra mágica para decir *zo mismo"* o *.estе mismo ordenador"* es `localhost`. Para decirle al navegador a qué puerto exacto debe llamar, añadimos dos puntos y el número de puerto de Windows (el de la izquierda de nuestra regla).

Escribe esto en tu navegador y pulsa Enter:

```
http://localhost:8080
```

Terminal 14: Dirección en tu navegador web

¡Magia! Deberías ver una pantalla de bienvenida que dice **"Welcome to nginx!"**.

Acabas de atravesar la caja blindada. Tu petición salió del navegador de Windows, llamó a la puerta 8080 de tu ciudad, cruzó el puente de Docker, entró por la ventanilla 80 del Food Truck, el cocinero de Nginx preparó la página web, y te la devolvió por el mismo camino.

[DOCKER Llamada a localhost:8080]

Si abres otro contenedor distinto, podrías mapearlo a otra puerta libre de tu Windows, por ejemplo: `-p 9000:80`. Así tendrías dos servidores web idénticos funcionando al mismo tiempo, uno en `localhost:8080` y otro en `localhost:9000`, sin que haya ningún conflicto entre ellos.

6. Almacenamiento: Cómo no perder tus datos

Hasta ahora hemos aprendido a pedirle al capataz que construya un Food Truck, lo encienda, le abra una ventanilla de pedidos (puertos) e incluso que lo destruya. Pero hay un pequeño detalle oscuro en la forma en que Docker construye estos camiones que debemos solucionar urgentemente.

6.1. El problema: Los contenedores son efímeros

Cuando el capataz lee el manual (Imagen) y construye el contenedor, crea un disco duro interno temporal para que la aplicación pueda funcionar.

El problema es que **los contenedores son amnésicos y de usar y tirar**. Si tú decides demoler el Food Truck (`docker rm`) para construir uno más moderno mañana, todo lo que estuviera dentro del camión se destruirá con él.

Imagina que has montado un Food Truck que funciona como Base de Datos. Durante una semana, has guardado los nombres y correos de 1.000 clientes. Si borras el contenedor, o si el contenedor se cuelga y se reinicia, **has perdido a todos tus clientes para siempre**.

Para evitar esta tragedia, necesitamos sacar la caja registradora del camión. Docker nos ofrece dos herramientas fundamentales para esto: los Volúmenes y los Bind Mounts.

6.2. Volúmenes: La caja fuerte gestionada por Docker

Un **Volumen** es un disco duro virtual externo. Siguiendo nuestra analogía, es una gran **caja fuerte ignífuga** que el capataz entierra bajo tierra, fuera del camión.

Cuando el capataz construye el Food Truck, tú le dices: *"Conecta el cajón de la registradora que está dentro del camión, directamente a esta caja fuerte externa"*. De esta forma, cuando el cocinero guarda un dato, cree que lo está guardando dentro de su camión, pero en realidad está cayendo por un tubo directamente a la caja fuerte indestructible.

Si mañana demueles el Food Truck, la caja fuerte sigue ahí. Cuando construyas un camión nuevo, solo tienes que conectarlo a la misma caja fuerte y ¡magia! Todos tus clientes siguen ahí.

Para crear este tubo, usamos el parámetro `-v` (de volumen). Al igual que con los puertos, la regla de oro es: **Izquierda (Nombre de la caja fuerte) : Derecha (Carpeta dentro del contenedor)**.

```
docker run -d -v mi_caja_fuerte:/ruta/de/la/base/de/datos nginx
```

Terminal 15: Crear un contenedor con un Volumen

Los Volúmenes son ideales para bases de datos (como PostgreSQL o MySQL) porque Docker los gestiona de forma hiper-eficiente. El único inconveniente es que para ti, desde Windows 11, es difícil abrir esa caja fuerte y cotillear los archivos a mano.

6.3. Bind Mounts: La cinta transportadora hacia tu Escritorio

¿Qué pasa si lo que quieres es crear una página web, editando el código desde tu editor favorito en Windows (como VS Code), y ver cómo los cambios aparecen inmediatamente en el servidor web de Docker?

Aquí entran en juego los **Bind Mounts**. En lugar de pedirle al capataz que cree una caja fuerte ciega, le dices: *"Quiero que conectes esta carpeta específica de mi Escritorio de Windows, directamente a la carpeta de la cocina del Food Truck"*.

Es como poner una **cinta transportadora de cristal** entre tu mesa de la oficina y la cocina del camión. Si tú pones una lechuga (un archivo HTML) en tu carpeta de Windows, el cocinero del contenedor la ve instantáneamente y la sirve. Si el camión explota, tus archivos siguen intactos en tu carpeta de Windows.

La sintaxis es idéntica (`-v`), pero a la izquierda, en lugar de poner un nombre inventado, ponemos la ruta real de tu Windows:

```
docker run -d -v C:/Users/TuUsuario/Escritorio/mi_web:/usr/share/nginx
    /html nginx
```

Terminal 16: Crear un contenedor con un Bind Mount

Nota importante: Los Bind Mounts son fantásticos para programar en el día a día, porque te permiten usar tus herramientas de Windows mientras la aplicación corre en el entorno perfecto de Linux dentro de Docker.

7. Docker Compose: La Receta Maestra

Ya sabemos iniciar contenedores, abrirles puertos y conectarles cajas fuertes. Pero a medida que tu negocio crezca, te darás cuenta de un nuevo problema.

7.1. ¿Por qué escribir comandos largos es mala idea?

Imagina que tu aplicación necesita un servidor web, una base de datos y un sistema de memoria caché. Para levantar todo eso cada mañana, tendrías que abrir tu PowerShell y gritarle al capataz una retahíla de comandos inmensa, recordando cada puerto, cada contraseña y cada volumen:

```
docker run -d -p 8080:80 -v mi_web:/html --name servidor nginx
docker run -d -p 3306:3306 -v datos_db:/var/lib/mysql -e
    MYSQL_ROOT_PASSWORD=secreta --name base_datos mysql
...
```

Terminal 17: El infierno de los comandos manuales

Esto es propenso a errores humanos (¿qué pasa si te olvidas de escribir un guion?). Además, ¿cómo le pasas todas estas instrucciones exactas a un compañero de trabajo para que monte su propia franquicia? ¿Por WhatsApp?

7.2. El archivo `docker-compose.yml`: Tu infraestructura como código

Para solucionar esto, Docker inventó **Docker Compose**.

En nuestra analogía, Compose es el **Plano Maestro del Centro Comercial**. En lugar de darle al capataz 50 órdenes sueltas a gritos, te sientas tranquilamente, redactas un documento elegante y estructurado con todo lo que necesitas, se lo entregas, y él se encarga de construir todo el complejo de golpe, conectando los camiones entre sí automáticamente.

Este documento siempre se llama `docker-compose.yml`. Está escrito en un lenguaje llamado YAML, que es muy fácil de leer para los humanos.

Vamos a ver cómo se redacta. Abre el bloc de notas (o tu editor de código) y crea un archivo con esta estructura, dentro de la carpeta que decidas (y que solo puede tener un archivo `docker-compose.yml`.

Fíjate en cómo definimos la *imagen*, los *puertos* y los *volúmenes* (lo mismo que hacíamos en consola, pero ahora de forma ordenada):

YAML 1: Plano Maestro de nuestro servidor web

```yaml
services:
    mi_servidor_web:
        image: nginx:latest
        container_name: web_principal
        ports:
            - "8080:80"
        volumes:
            - mi_caja_fuerte:/usr/share/nginx/html
        restart: always

volumes:
    mi_caja_fuerte:
```

¡Cuidado con los espacios!

En el lenguaje YAML, se utilizan los **espacios en blanco** (indentación) para meter un elemento dentro de otro.

7.3. Levantar y tirar todo el sistema con un solo comando

Una vez que guardas ese archivo `docker-compose.yml` en una carpeta, la magia de verdad comienza. Abre tu PowerShell, navega hasta esa carpeta y escribe este único comando:

```
docker compose up -d
```

Terminal 18: Construir el Centro Comercial entero

El capataz leerá el documento, descargará las imágenes necesarias, creará los volúmenes, abrirá los puertos, construirá los contenedores y los dejará funcionando de fondo. ¡Todo con un comando de cuatro palabras!

Y lo mejor de todo: cuando termine tu jornada laboral y quieras apagar el servidor, borrar el contenedor y limpiar la red (conservando intacta la caja fuerte), solo tienes que decirle al capataz que desmonte la feria:

```
docker compose down
```

Terminal 19: Desmontar la infraestructura

Gracias a Docker Compose, la famosa frase *.En mi máquina funcionaba"* muere definitivamente. Si le envías ese pequeño archivo de texto a tu compañero de trabajo, él solo tendrá que hacer `docker compose up -d` y tendrá exactamente el mismo centro comercial funcionando en su Windows 11 en cuestión de segundos.

8. Práctica Final: Montando un Servidor Web Real

Ha llegado el momento de la verdad. Ya no vamos a usar la página web por defecto que trae Nginx. Vamos a abrir nuestra propia cafetería, con nuestro propio menú, y vamos a dejar que el capataz de Docker se encargue de toda la infraestructura.

8.1. Objetivo: Tu página web en el navegador

Vamos a crear una página web muy sencilla en Windows, y usaremos **Docker Compose** para levantar un servidor que la muestre al mundo a través de un **Bind Mount** (la cinta transportadora de cristal que vimos en el Apartado 6).

8.2. Paso 1: Crear el archivo HTML en Windows (Tu receta)

Primero, necesitamos un lugar de trabajo en nuestra ciudad (Windows 11).

1. Ve a tu Escritorio y crea una carpeta nueva llamada `mi_primera_web`.
2. Entra en la carpeta, haz clic derecho y crea un nuevo *Documento de texto*.
3. Ábrelo con el Bloc de notas y pega el siguiente código web:

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Mi Food Truck</title>
    <style>
        body { text-align: center; padding: 50px; font-family: Arial; }
        h1 { color: navy; }
    </style>
</head>
<body>
    <h1>¡Bienvenidos a mi primera web con Docker!</h1>
    <p>Si estás leyendo esto, el capataz ha hecho bien su trabajo.</p>
</body>
</html>
```

4. Guarda el archivo y cámbiale el nombre a **`index.html`**.

   Asegúrate de que no se llame `index.html.txt`. Si no ves las extensiones en Windows, ve al menú *Ver* del explorador de archivos y marca *Extensiones de nombre de archivo*.

8.3. Paso 2: Configurar el Plano Maestro (docker-compose.yml)

Ahora vamos a darle las instrucciones de construcción al capataz. En esa misma carpeta llamada`mi_primera_web`, crea otro archivo de texto y llámalo **`docker-compose.yml`**.

Ábrelo con el Bloc de notas y escribe exactamente esto:

YAML 2: Plano maestro de nuestra web

```yaml
services:
  cafeteria_web:
```

```
    image: nginx:latest
    container_name: mi_servidor_nginx
    ports:
      - "8080:80"
    volumes:
      - ./:/usr/share/nginx/html
```

El truco del punto y la barra (./):

En la sección *volumes*, hemos puesto `./` a la izquierda de los dos puntos. En el idioma de la terminal, esto significa *"usa la carpeta exacta en la que estoy ahora mismo"*. Así nos ahorramos escribir toda la ruta larga de `C:\Users\...\Escritorio\mi_primer\_web`.

8.4. Paso 3: Conectar todo y ver el resultado

Ya tenemos los ingredientes (el HTML) y los planos de construcción (el YAML). Solo falta darle la orden al capataz.

1. Abre tu terminal de **PowerShell**.
2. Navega hasta la carpeta que has creado usando el comando `cd` (Change Directory).

   Si la creaste en el escritorio, el comando será algo así:

```
cd ~\Desktop\mi_primera_web
```

Terminal 20: Navegar a la carpeta

3. Una vez dentro de la carpeta, ejecuta el comando mágico de construcción:

```
docker compose up -d
```

Terminal 21: Levantar la infraestructura descrita en `docker-compose.yml`

Verás cómo Docker descarga Nginx (si no lo tenía ya), crea la red, y arranca el contenedor. Ahora, el momento de la verdad. Abre tu navegador web y entra en la dirección de la ventanilla de tu camión:

`http://localhost:8080`

¡La prueba definitiva! Sin apagar nada, abre tu archivo `index.html` en el Bloc de notas de Windows, cambia el texto por *"¡He modificado la web en directo!"*, guarda el archivo, y recarga la página en tu navegador (F5). Verás que el cambio es inmediato. Eso es la magia del Bind Mount: estás editando en Windows y ejecutando en el corazón de Linux.

Para terminar, vuelve a PowerShell, a la carpeta donde están definidos los servicios en tu archivo `docker-compose.yml`, que dijimos que era `~\Desktop\mi_primera_web`, y desmonta la cafetería limpiamente:

```
docker compose down
```

Terminal 22: Apagar y limpiar todo

9. Anexos y Chuletas

9.1. Chuleta de comandos esenciales de supervivencia

Imprime esta página y pégala cerca de tu monitor. Estos son los comandos que usarás el 99 % de tu tiempo como "inversor" de franquicias:

Comando	¿Qué hace en la vida real? (Analogía)
`docker run -d imagen`	**Construir y abrir** Descarga el manual, construye el Food Truck y lo deja abierto trabajando en segundo plano.
`docker ps`	**Pasar lista** Muestra qué camiones están ahora abiertos y trabajando.
`docker ps -a`	**Inventario total** Muestra todos tus camiones, tanto los abiertos como los que están cerrados/aparcados.
`docker stop nombre`	**Cerrar por hoy** Cierra la persiana del camión temporalmente para ahorrar batería de tu PC.
`docker start nombre`	**Abrir por la mañana** Vuelve a encender un camión que estaba detenido.
`docker rm nombre`	**Demolición** Destruye el camión para siempre (perderás lo de dentro si no usaste un Volumen).
`docker compose up -d`	**Construir el Centro Comercial** Lee el archivo YAML y monta toda tu infraestructura de golpe.
`docker compose down`	**Desmontar el Centro Comercial** Apaga los contenedores del YAML, dejando tu PC limpio.

Tabla 1: Comandos básicos de Docker

9.2. Solución de problemas comunes en Windows 11

A veces, el capataz tiene un mal día. Aquí tienes las soluciones a los dos errores más típicos que te vas a encontrar:

9.2.1. Error 1: "bind: address already in use"

- **El síntoma:** Al hacer `docker run` o `docker compose up`, sale un texto rojo quejándose de la dirección.
- **El problema:** Le has dicho al capataz que conecte el camión a la puerta `8080` de tu Windows, pero ya hay otro programa (u otro contenedor que olvidaste apagar) usando esa puerta.
- **La solución:** Abre otro puerto distinto en tu máquina (por ejemplo, cambia el YAML a `"8081:80"`) o busca qué contenedor está usando el puerto antiguo con `docker ps` y apágalo con `docker stop`.

9.2.2. Error 2: Docker Desktop se queda en "Starting..." y nunca arranca

- **El síntoma:** El icono de la ballena se queda en amarillo o rojo y nunca se pone verde. La terminal dice *"error during connect"*.
- **El problema:** El barrio de Linux (WSL2) de tu Windows se ha colgado y el capataz no puede entrar a trabajar.
- **La solución:** No hace falta reiniciar el ordenador entero.

  Abre PowerShell como administrador, escribe `wsl -shutdown` (esto apaga todo Linux de golpe) y vuelve a abrir Docker Desktop.

  Se reiniciará limpio y en segundos.